

InClass - Documentación Técnica y de Proyecto

1. Introducción

InClass es un sistema integral de control de asistencia basado en reconocimiento facial. Su objetivo es automatizar y asegurar el registro de asistencia en entornos educativos, reduciendo el tiempo perdido en el pase de lista manual y evitando suplantaciones de identidad.

El sistema distingue dos roles principales:

- **Docente:** Puede crear sesiones de clase, iniciar la toma de asistencia y revisar reportes consolidados.
 - **Estudiante:** Puede ver su historial de asistencia y recibir notificaciones (vía Firebase/Email) cuando se registra su presencia.
-

2. Arquitectura del Sistema

El sistema sigue una arquitectura cliente-servidor, componiéndose de los siguientes elementos:

- **Frontend (Cliente):** Aplicación móvil desarrollada en **Flutter** (Dart), compatible con Android/iOS.
 - **Backend (API Rest):** Desarrollado en **Python** utilizando **FastAPI**. Provee los endpoints de comunicación.
 - **Motor de Reconocimiento Facial:** Utiliza la librería `insightface` con el modelo `buffalo_l` para la extracción y comparación de *embeddings* faciales.
 - **Base de Datos Principal:** Se utiliza **Supabase** (PostgreSQL) para gestionar usuarios, cursos, sesiones y registros de asistencia.
 - **Base de Datos Vectorial (Opcional):** Posibilidad de usar **Pinecone** para almacenar y buscar *embeddings* faciales de manera escalable.
 - **Servicios Externos:**
 - **Resend:** Envío de correos electrónicos transaccionales.
 - **Firebase Cloud Messaging (FCM):** Envío de notificaciones push en tiempo real a los dispositivos móviles.
-

3. Backend (API & Motor Facial)

3.1. Tecnologías principales

- **Framework:** FastAPI
- **Reconocimiento Facial:** Insightface (`buffalo_l`), OpenCV, Numpy
- **ORM / BD:** Supabase (Client SDK)

- **Autenticación:** JWT (JSON Web Tokens)

3.2. Estructura de Directorios

- `main.py`: Punto de entrada de la aplicación FastAPI. Configura CORS y registra todos los enrutadores (routers).
- `config.py`: Centraliza las configuraciones mediante variables de entorno (`.env`), incluyendo las claves de Supabase, JWT, Pinecone y parámetros de tolerancia para el modelo facial (Ej. `MIN_CONFIDENCE = 0.80`).
- `database.py`: Instancia el cliente de Supabase.
- `models/`: Contiene los esquemas de Pydantic (`schemas.py`) para la validación de los datos de entrada/salida de la API.
- `routes/`: Separa los endpoints lógicamente:
 - `auth.py`: Inicio de sesión y emisión de tokens.
 - `courses.py`: Gestión de cursos asignados a docentes y estudiantes.
 - `sessions.py`: Creación y control de sesiones de clase.
 - `students.py`: Registro de estudiantes y enrolamiento facial.
 - `attendance.py`: Recepción de frames/imágenes de la clase y procesamiento facial para asistencia.
 - `reports.py`: Generación de informes de asistencia.
- `services/`: Lógica de integración con terceros (`notification_service.py` para Firebase FCM y correos).
- `face_engine/`: Módulo de Inteligencia Artificial.
 - `recognizer.py`: Inicializa el modelo `buffalo_l`, extrae *embeddings* y calcula la similitud coseno (`cosine_similarity`).
 - `vector_db.py`: Integración opcional con Pinecone para consultas vectoriales.

3.3. Algoritmo de Reconocimiento

1. La cámara envía un *frame* (imagen) al endpoint correspondiente.
 2. `FaceRecognizer` procesa el frame extrayendo los rostros y sus vectores (*embeddings* de 512 dimensiones).
 3. Se compara el vector extraído con la base de datos de estudiantes en memoria (o en Pinecone) calculando la distancia coseno.
 4. Si la coincidencia supera el `MIN_CONFIDENCE` (80%), se identifica al estudiante y se marca su asistencia en Supabase.
-

4. Frontend (Aplicación Móvil)

4.1. Tecnologías principales

- **Framework:** Flutter
- **Lenguaje:** Dart
- **Notificaciones:** Firebase Cloud Messaging (FCM)
- **Almacenamiento Local:** `shared_preferences` para almacenar tokens JWT de sesión.

4.2. Estructura de Directorios (lib/)

- `main.dart`: Configura Firebase, maneja el *Splash Screen*, solicita permisos de notificación y enruta al usuario dependiendo de si ya existe una sesión guardada (`DocenteHomeScreen` o `EstudianteHomeScreen`).
 - `screens/`: Pantallas de la aplicación.
 - `login_screen.dart`: Formulario de inicio de sesión.
 - `docente/`: Vistas exclusivas para el profesor (gestión de cursos, apertura de cámara, panel de asistencia en vivo).
 - `estudiante/`: Vistas exclusivas para el alumno (perfil, estado de asistencia en sus materias).
 - `services/`:
 - `api_service.dart`: Encapsula las peticiones HTTP (GET, POST) hacia el backend en FastAPI (enviando el JWT en las cabeceras).
 - `theme/`:
 - `app_theme.dart`: Centraliza colores (ej. azul institucional #1F3864), tipografías y estilos de botones.
-

5. Modelos de Datos (Supabase)

Aunque los esquemas de base de datos están gestionados desde Supabase, el backend los refleja en `schemas.py`:

- **Usuario (Docentes/Estudiantes)**: Identificador, Nombre, Código institucional, Contraseña cifrada, Rol.
 - **Cursos**: ID, Nombre, Docente Asignado.
 - **Matrículas (Enrollments)**: Relación N:M entre Estudiantes y Cursos.
 - **Sesiones**: Clase particular creada por un docente (Día, Hora de inicio, Curso asociado, Estado Abierto/Cerrado).
 - **Asistencia**: Registro único que relaciona a un Estudiante, una Sesión, y un *timestamp* de confirmación.
-

6. Flujo de Funcionamiento (Ejemplo Práctico)

1. **Inicio de Sesión**: Docente inicia sesión en la app Flutter. El backend valida en Supabase y retorna un JWT.
2. **Crear Sesión**: Docente selecciona su curso e inicia una sesión. Se crea un registro en la BD.
3. **Monitoreo Facial**: La cámara del dispositivo docente comienza a capturar imágenes y enviarlas intermitentemente al endpoint de reconocimiento en FastAPI.
4. **Procesamiento IA**: `insightface` detecta un rostro. El backend calcula su similitud coseno con la base de datos local `embeddings.pkl` o Pinecone.
5. **Registro y Notificación**:
 - Al detectar a "Juan Pérez" con 85% de similitud, se registra su asistencia en Supabase.
 - El backend envía una notificación push vía Firebase al celular de Juan Pérez indicando: "Tu asistencia ha sido registrada exitosamente".

6. **Fin de Clase:** El docente finaliza la sesión en la app, cerrando la posibilidad de más registros y visualizando el porcentaje final de asistencia.